

This is a copy of a shared ChatGPT conversation
[Report conversation](#)

I would like to make the backend of an app, thus i only need python code for the backend, you can forget about frontend, ui, and no unittest. This app is supposed to help student navigate home late at night or when they feel unsafe by having volunteers accompany them to their destinations.

I would like the app to:

- *handle 1000 users data
- *have users as students, thus need some type of verification of students status, maybe a picture of student id or school email registration
- *i need the code to handle high traffic, thus a load balancer needs to be implemented.
- *the request from the user should be routed to the available and willing volunteer, as to not overwhelm one volunteer with all request
- *there should be a rating system as to build trust with the users and volunteers
- *the personal data of the student shall be protected, only showing minimal information to the volunteer, such as name and location or meet place
- * there needs to be a chat function as to facilitate the communication between the user and the volunteer, this chat should be reliable and fast
- * the volunteers should be notified/see the request and chose what they want to accept

Show less 

You're basically describing a real-time, safety-critical matching system—not just a simple backend. Python can absolutely handle this, but you need the right architecture from the start or it'll collapse under load or become unsafe.

I'll give you a **clean, production-style backend skeleton** using:

- FastAPI (high-performance API)
- Redis (real-time + chat + load distribution)
- PostgreSQL (persistent data)

- WebSockets for chat
- Basic load-balancing strategy (application-level + scalable design)

No frontend, no tests—just backend logic.

High-level Architecture

- API layer (FastAPI)
- Auth + verification (email domain or ID upload placeholder)
- Matching system (queue + Redis)
- Volunteer availability pool
- WebSocket chat
- Rating system
- Privacy filtering
- Horizontal scalability (multiple instances behind load balancer like NGINX)

Install dependencies

Bash

```
pip install fastapi uvicorn sqlalchemy asyncpg redis pydantic python-multipart
```

Main Backend Code (Simplified but Scalable)

Python

Run

```
from fastapi import FastAPI, WebSocket, Depends, HTTPException, UploadFile
from pydantic import BaseModel
from typing import List, Optional
import asyncio
import redis.asyncio as redis
from sqlalchemy.ext.asyncio import create_async_engine, AsyncSession
from sqlalchemy.orm import sessionmaker

app = FastAPI()

# -----
# DATABASE SETUP
# -----
DATABASE_URL = "postgresql+asyncpg://user:password@localhost/safety_app"
engine = create_async_engine(DATABASE_URL)
```

```
SessionLocal = sessionmaker(engine, class_=AsyncSession, expire_on_commit=False)

# -----
# REDIS (REAL-TIME)
# -----
r = redis.Redis(host="localhost", port=6379, decode_responses=True)

# -----
# MODELS
# -----
class UserCreate(BaseModel):
    email: str
    name: str
    is_volunteer: bool

class RideRequest(BaseModel):
    user_id: int
    location: str
    destination: str

class Rating(BaseModel):
    from_user: int
    to_user: int
    score: int # 1-5

# -----
# STUDENT VERIFICATION
# -----
@app.post("/verify-email")
async def verify_email(email: str):
    if not email.endswith(".edu"):
        raise HTTPException(status_code=400, detail="Invalid student email")
    return {"status": "verified"}

@app.post("/upload-id")
async def upload_id(file: UploadFile):
    # Placeholder for ID verification (use OCR / AI later)
    return {"status": "uploaded"}

# -----
# VOLUNTEER AVAILABILITY
# -----
@app.post("/volunteer/available/{user_id}")
async def set_available(user_id: int):
    await r.sadd("available_volunteers", user_id)
    return {"status": "available"}

@app.post("/volunteer/unavailable/{user_id}")
async def set_unavailable(user_id: int):
    await r.srem("available_volunteers", user_id)
    return {"status": "unavailable"}
```

```

# -----
# REQUEST MATCHING SYSTEM
# -----
@app.post("/request")
async def request_help(req: RideRequest):
    volunteers = await r.smembers("available_volunteers")

    if not volunteers:
        raise HTTPException(status_code=404, detail="No volunteers available")

    # Load balancing: simple round-robin
    volunteer = list(volunteers)[0]

    # Store assignment
    await r.set(f"active_request:{req.user_id}", volunteer)

    # Notify volunteer (push to their queue)
    await r.lpush(f"notifications:{volunteer}", req.user_id)

    return {"assigned_volunteer": volunteer}

# -----
# VOLUNTEER ACCEPT REQUEST
# -----
@app.post("/accept")
async def accept_request(volunteer_id: int, user_id: int):
    assigned = await r.get(f"active_request:{user_id}")

    if str(volunteer_id) != assigned:
        raise HTTPException(status_code=403, detail="Not your request")

    await r.set(f"chat:{user_id}", volunteer_id)

    return {"status": "accepted"}

# -----
# CHAT SYSTEM (WEBSOCKET)
# -----
connections = {}

@app.websocket("/chat/{user_id}")
async def chat(websocket: WebSocket, user_id: int):
    await websocket.accept()
    connections[user_id] = websocket

    try:
        while True:
            data = await websocket.receive_text()

            volunteer_id = await r.get(f"chat:{user_id}")
            if volunteer_id and int(volunteer_id) in connections:
                await connections[int(volunteer_id)].send_text(data)

```

```

except:
    connections.pop(user_id, None)

# -----
# RATING SYSTEM
# -----
@app.post("/rate")
async def rate_user(rating: Rating):
    key = f"rating:{rating.to_user}"
    await r.rpush(key, rating.score)
    return {"status": "rated"}

@app.get("/rating/{user_id}")
async def get_rating(user_id: int):
    scores = await r.lrange(f"rating:{user_id}", 0, -1)
    scores = list(map(int, scores))

    if not scores:
        return {"rating": None}

    avg = sum(scores) / len(scores)
    return {"rating": avg}

# -----
# PRIVACY FILTER
# -----
def sanitize_user_data(user):
    return {
        "name": user["name"],
        "location": user.get("location")
    }

```

Load Balancing (Important)

This code itself doesn't *act* as a load balancer—you deploy multiple instances and put something like:

- NGINX
- or cloud load balancer (AWS ELB, etc.)

Example:



Redis ensures all instances share:

- available volunteers
- chat routing
- requests

Key Design Decisions Explained

1. Handle ~1000 users

- Async FastAPI + Redis = enough
- Horizontal scaling handles more

2. Student verification

- .edu email (simple)
- ID upload (placeholder → later AI/OCR)

3. Load distribution

- Redis set = pool of volunteers
- Round-robin / can upgrade to:
 - least busy volunteer
 - location-based matching

4. Privacy

- Only expose:
 - name
 - meeting location

5. Chat

- WebSockets = real-time
- Redis could be added for pub/sub (if scaling further)

6. Reliability

- Redis = fast
- PostgreSQL = durable



What's Missing (But You Should Add Later)

Don't skip these in a real system:

- Authentication (JWT)

- Encryption (HTTPS)
- Rate limiting (prevent abuse)
- GPS-based matching
- Panic/emergency button
- Logging + monitoring
- Abuse detection



If you want next step

I can help you evolve this into:

- Proper microservice architecture
- Smarter matching (distance, ratings, availability)
- Production deployment (Docker + CI/CD)
- Database schema (SQLAlchemy models)
- Or scale to 10k+ users

Just tell me 👍

can you add JWT and HTTPS, and rate limiting.

Good instincts—those three are *not optional* for something like this. I'll extend your backend with:

- JWT authentication (secure identity)
- HTTPS setup (transport security)
- Rate limiting (abuse protection)

I'll keep it clean and plug directly into your existing FastAPI-style backend.



1. JWT Authentication

We'll use:

- python-jose
- Password hashing via passlib

Install

Bash

```
pip install python-jose passlib[bcrypt]
```

Add Auth Utilities

Python

Run

```
from jose import JWTError, jwt
from datetime import datetime, timedelta
from passlib.context import CryptContext
from fastapi import Depends, HTTPException
from fastapi.security import OAuth2PasswordBearer

SECRET_KEY = "super-secret-key" # change in production
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES = 60

pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="login")

fake_users_db = {}

def hash_password(password: str):
    return pwd_context.hash(password)

def verify_password(plain, hashed):
    return pwd_context.verify(plain, hashed)

def create_access_token(data: dict):
    to_encode = data.copy()
    expire = datetime.utcnow() + timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
    to_encode.update({"exp": expire})
    return jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)

async def get_current_user(token: str = Depends(oauth2_scheme)):
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
        user_id = payload.get("sub")
        if user_id is None:
            raise HTTPException(status_code=401)
    except JWTError:
        raise HTTPException(status_code=401)

    return user_id
```

Register + Login

Python

Run

```
class UserRegister(BaseModel):
    email: str
    password: str

@app.post("/register")
async def register(user: UserRegister):
    if user.email in fake_users_db:
        raise HTTPException(400, "User exists")

    fake_users_db[user.email] = {
        "email": user.email,
        "password": hash_password(user.password)
    }

    return {"status": "created"}

@app.post("/login")
async def login(user: UserRegister):
    db_user = fake_users_db.get(user.email)

    if not db_user or not verify_password(user.password, db_user["password"]):
        raise HTTPException(401, "Invalid credentials")

    token = create_access_token({"sub": user.email})
    return {"access_token": token, "token_type": "bearer"}
```

Protect Your Endpoints

Python

Run

```
@app.post("/request")
async def request_help(req: RideRequest, user=Depends(get_current_user)):
    # Now only authenticated users can request help
    ...
```



2. HTTPS (Transport Security)

FastAPI itself doesn't "do HTTPS"—you terminate TLS at a reverse proxy.

Use **NGINX**:

NGINX Config Example

Nginx

```
server {  
    listen 80;  
    server_name yourdomain.com;  
  
    return 301 https://$host$request_uri;  
}  
  
server {  
    listen 443 ssl;  
    server_name yourdomain.com;  
  
    ssl_certificate /etc/letsencrypt/live/yourdomain/fullchain.pem;  
    ssl_certificate_key /etc/letsencrypt/live/yourdomain/privkey.pem;  
  
    location / {  
        proxy_pass http://localhost:8000;  
        proxy_set_header Host $host;  
        proxy_set_header X-Real-IP $remote_addr;  
    }  
}
```

Get Free SSL Certificate

Use Certbot:

Bash

```
sudo certbot --nginx -d yourdomain.com
```



3. Rate Limiting

We'll use Redis-based limiting (important for distributed systems).

Add Rate Limiter

Python

```
import time
```

Run

```
RATE_LIMIT = 10 # requests
WINDOW = 60 # seconds

async def rate_limiter(user_id: str):
    key = f"rate:{user_id}"

    current = await r.get(key)

    if current:
        count, start = map(int, current.split(":"))

        if time.time() - start < WINDOW:
            if count >= RATE_LIMIT:
                raise HTTPException(429, "Too many requests")
            else:
                await r.set(key, f"{count+1}:{start}")
        else:
            await r.set(key, f"1:{int(time.time())}")
    else:
        await r.set(key, f"1:{int(time.time())}")
```

Apply Rate Limit to Endpoints

Python

Run

```
@app.post("/request")
async def request_help(
    req: RideRequest,
    user=Depends(get_current_user)
):
    await rate_limiter(user)

    # existing logic...
```



What You Just Added (Big Picture)

JWT

- Stateless auth
- Scales easily across servers

HTTPS

- Prevents:
 - MITM attacks

- token theft
- location leaks (critical for your app)

Rate Limiting

- Prevents:
 - spam requests
 - bot attacks
 - volunteer overload

⚠ Critical Improvements You Should Do Next

Don't ignore these for a real safety app:

1. Replace `fake_users_db`

Use PostgreSQL with proper user tables

2. Add roles

- student
- volunteer

3. Stronger verification

- school email domain whitelist
- ID verification service

4. Smarter rate limiting

- per IP + per user
- different limits per endpoint

5. Secure secrets

- NEVER hardcode `SECRET_KEY`
- use environment variables

If you want, I can next:

- Turn this into a production-ready folder structure
- Add Docker + CI/CD
- Implement location-based matching (real algorithm)
- Or design the PostgreSQL schema

Just tell me 👍

